01_Analisis_Impedancia_VFT

April 10, 2026

1 Reporte 01: Discretización Espacial y Asignación de Impedancia Temporal (VFT)

1.1 01.- Marco Teórico

En el analisis de las redes de transporte Urbano, la topología estática no refleja la realidad operativa. Para resolver esto, el presente modelo inyecta atributos dinámicos a cada vector de la red ($\vec{uv}$) utilizando la **Ecuación VFT**.
El tiempo de traslado continuo (T_t) de un segmento geodésico se discretiza matemáticamente mediante la siguiente expresión:

$$T_t = \left(\frac{D}{V}\right) \times C_f$$

Donde: * D: Distancia esférica del segmento en metros (calculada vía Haversine). * V: Velocidad comercial imputada en m/min. * C_f: Coeficiente de Fricción Vial. Calculado en función del nivel de permeabilidad del derecho de vía (e.g., mixto, confinado, exclusivo) y ponderado por un factor de saturación característico de la CDMX.

Adicionalmente, el modelo aísla la "paradoja de la espera fantasma" segregando el Costo de Abordaje (T_b), el cual será evaluado exclusivamente en los nodos de transferencia intermodal:

$$T_b = \frac{F}{2}$$

(Donde F es la frecuencia de paso de los vehículos en minutos).

[2]:
```
# Habilitar recarga automática para reflejar cambios en 'src' sin reiniciar el␣
↪notebook
%load_ext autoreload
%autoreload 2

import sys
import os

proyecto_path = os.path.abspath('..')
if proyecto_path not in sys.path:
    sys.path.append(proyecto_path)
```

```
print(f"Ruta del proyecto cargada: {proyecto_path}")
```

```
Ruta del proyecto cargada: /home/galigaribaldi/Documentos/Code/VFTModel
```

[3]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
## Modulos propios
from src.api.schemas.schemas import GeoJSONTransportSchema
from src.infrastructure.go_client.client import fetch_full_network
from src.core.services.graph_builder import VFTGraphBuilder

raw_data = await fetch_full_network()

validated_payload = GeoJSONTransportSchema(**raw_data)

# 3. Construcción topológica y cálculo de Impedancia VFT
GRAF= VFTGraphBuilder(validated_payload)
grafo_vft = GRAF.build_graph()

print(f"\nResumen de la Red Analizada:")
print(f"Nodos (Estaciones/Vértices): {grafo_vft.number_of_nodes()}")
print(f"Aristas (Vectores de flujo): {grafo_vft.number_of_edges()}")
```

```
2026-04-09 23:54:40 | INFO     | VFT_Model | Construyendo el puente hacia el
módulo espacial de Go…
2026-04-09 23:54:40 | INFO     | VFT_Model | Solicitando capa espacial a:
http://localhost:8080/movilidad/mapas/geojsonEstacion
2026-04-09 23:54:40 | INFO     | VFT_Model | Solicitando capa espacial a:
http://localhost:8080/movilidad/mapas/geojsonLinea
2026-04-09 23:54:40 | INFO     | VFT_Model | Red extraída: 22785 entidades
espaciales listas para VFT.
2026-04-09 23:54:41 | INFO     | VFT_Model | Iniciando VFTGraphBuilder en modo:
REALISTIC_INTEGRATION
2026-04-09 23:54:41 | INFO     | VFT_Model | Fase 1 y 2: Extrayendo nodos y
trazos base…
2026-04-09 23:54:41 | INFO     | VFT_Model | Grafo Base construido: 101621 Nodos
y 93816 Segmentos.
2026-04-09 23:54:41 | INFO     | VFT_Model | Fase 3: Integrando sistemas
(Tolerancia peatonal: 85.0m)…
2026-04-09 23:55:08 | INFO     | VFT_Model | Se crearon 20482 aristas de
Transbordo Peatonal.
2026-04-09 23:55:08 | INFO     | VFT_Model | Aplicando Motor de Impedancia…
2026-04-09 23:55:08 | INFO     | VFT_Model | Iniciando inyección de motor de
impendancia sobre VFT:
2026-04-09 23:55:08 | INFO     | VFT_Model | Impedancia aplicada exitosamente a
114298 segmentos.
```

```
Resumen de la Red Analizada:
Nodos (Estaciones/Vértices): 101621
Aristas (Vectores de flujo): 114298
```

1.2 2. Análisis Estadístico de la Impedancia Inyectada

A continuación, extraemos los atributos multidimensionales calculados en el modelo `VFTImpedanceModel` para validar su coherencia física frente a los distintos sistemas de transporte.

[4]:
```python
# Extraer las propiedades de todas las aristas
edges_data = []
for u, v, data in grafo_vft.edges(data=True):
    # Filtrar solo aristas que tengan atributos calculados (ignorar␣
 ↪transferencias peatonales por ahora)
    if 'travel_time_min' in data:
        edges_data.append({
            'Origen_Lon': u[0],
            'Origen_Lat': u[1],
            'Sistema': data.get('sistema', 'Desconocido'),
            'Derecho_Via': data.get('derecho_de_via', 'mixto'),
            'Distancia_m': data.get('distancia_segmento_m', 0),
            'Friccion_Cf': data.get('coeficiente_friccion', 1.0),
            'Tiempo_Viaje_min': data.get('travel_time_min', 0.0),
            'Costo_Abordaje_min': data.get('boarding_cost_min', 0.0)
        })

df_edges = pd.DataFrame(edges_data)

# Mostrar las primeras filas del conjunto analítico
df_edges.head()
```

[4]:
```
   Origen_Lon  Origen_Lat              Sistema Derecho_Via  Distancia_m  \
0  -99.223269   19.364305  Transbordo Peatonal       mixto        51.08
1  -99.223269   19.364305  Transbordo Peatonal       mixto        60.86
2  -99.223269   19.364305  Transbordo Peatonal       mixto        69.44
3  -99.288580   19.333000  Transbordo Peatonal       mixto        27.26
4  -99.288580   19.333000  Transbordo Peatonal       mixto        82.10

   Friccion_Cf  Tiempo_Viaje_min  Costo_Abordaje_min
0          1.8            0.3678                 7.5
1          1.8            0.4382                 7.5
2          1.8            0.5000                 7.5
3          1.8            0.1963                 7.5
4          1.8            0.5911                 7.5
```

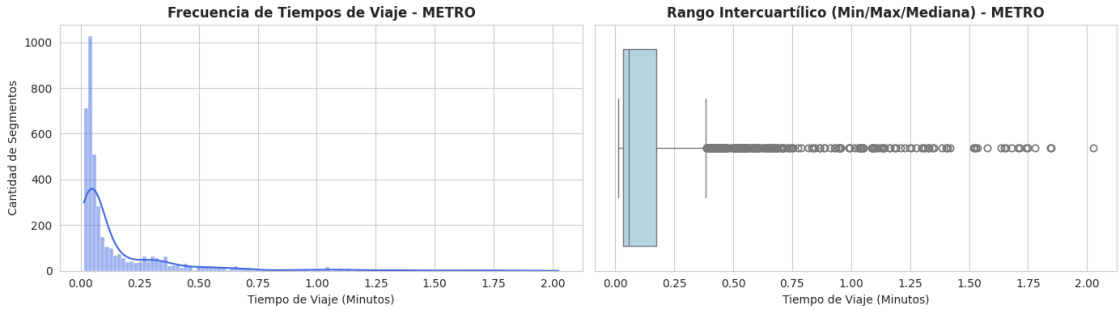
1.3 3. Glosario de Indicadores Analíticos (Modelo VFT)

Para facilitar la interpretación de los resultados estadísticos obtenidos de la red, a continuación se definen métricas clave evaluadas por el algoritmo de impedancia:

- **Segmentos_Analizados:** Representa el número total de vectores o "calles" físicas que componen la topología del sistema. No es la longitud total de la ruta, sino en cuántos fragmentos de esquina a esquina cortó el modelo a la ciudad.
- **Distancia_Promedio_m:** La longitud media (en metros) de cada segmento topológico. Calculada de nodo a nodo mediante geometría esférica (Fórmula de Haversine).
- **Friccion_Promedio (C_f):** El nivel de castigo temporal que sufre el transporte por interactuar con la ciudad. Un valor de `1.0` indica flujo ininterrumpido (ej. un túnel del Metro), mientras que valores cercanos a `1.8` indican un fuerte embotellamiento por tráfico mixto.
- **Tiempo_Promedio_min:** El tiempo neto estimado en minutos que le toma al vehículo cruzar un segmento promedio. Es el resultado dinámico de procesar la distancia, la velocidad comercial y la fricción vial (C_f).

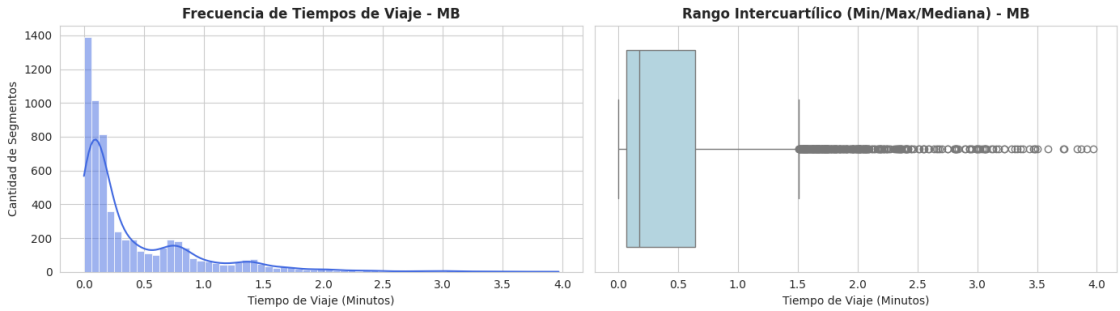
[7]:
```
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, Markdown

# Configuración visual global para las gráficas
sns.set_style("whitegrid")

# 1. Separar el DF masivo en DataFrames individuales por sistema
sistemas_principales = ['METRO', 'MB', 'RTP', 'CC', 'CBB',
                        'TROLE', 'TL', 'MEXICABLE', 'MEXIBÚS']

# Creamos un diccionario para acceder a ellos fácilmente
dfs_por_sistema = {}
for sistema in sistemas_principales:
    dfs_por_sistema[sistema] = df_edges[df_edges['Sistema'] == sistema].copy()

# 2. Análisis iterativo por sistema: Gráficas y Percentiles
for sistema in sistemas_principales:
    df_sis = dfs_por_sistema[sistema]

    # Filtro opcional: Limitamos a segmentos < 1000m solo para que la gráfica␣
↪no se deforme
    # por valores atípicos (ej. un cablebús de 3km sin paradas). La tabla sí␣
↪usa todos los datos.
    df_sis_plot = df_sis[df_sis['Distancia_m'] < 1000]

    # Crear figura con dos paneles: Histograma de Tiempos y Boxplot de Cuartiles
    fig, axes = plt.subplots(1, 2, figsize=(14, 4))

    # Panel Izquierdo: Curva de Distribución (Cómo se agrupan los tiempos)
```

```python
    sns.histplot(data=df_sis_plot, x='Tiempo_Viaje_min', kde=True,␣
↪color='royalblue', ax=axes[0])
    axes[0].set_title(f'Frecuencia de Tiempos de Viaje - {sistema}',␣
↪fontsize=12, fontweight='bold')
    axes[0].set_xlabel('Tiempo de Viaje (Minutos)')
    axes[0].set_ylabel('Cantidad de Segmentos')

    # Panel Derecho: Boxplot (Muestra Mínimo, Máximo y Cuartiles)
    sns.boxplot(data=df_sis_plot, x='Tiempo_Viaje_min', color='lightblue',␣
↪ax=axes[1])
    axes[1].set_title(f'Rango Intercuartílico (Min/Max/Mediana) - {sistema}',␣
↪fontsize=12, fontweight='bold')
    axes[1].set_xlabel('Tiempo de Viaje (Minutos)')

    plt.tight_layout()
    plt.show()

    # 3. Generar la Tabla Estadística Descriptiva (Promedio, Max, Min,␣
↪Percentiles)
    # Pandas .describe() ya calcula los percentiles 25%, 50% (mediana) y 75%
    stats = df_sis[['Distancia_m', 'Friccion_Cf', 'Tiempo_Viaje_min']].
↪describe().T

    # Filtramos y renombramos las columnas para que sean claras y académicas
    stats = stats[['mean', 'min', '25%', '50%', '75%', 'max']]
    stats.columns = ['Promedio', 'Mínimo', 'Cuartil 25%', 'Mediana (50%)',␣
↪'Cuartil 75%', 'Máximo']

    # Mostrar el título y la tabla renderizada
    display(Markdown(f"### Resumen de Impedancia y Estadísticas:␣
↪**{sistema}**"))
    display(stats.round(2)) # Redondeamos a 2 decimales para limpieza
    print("=" * 100) # Línea divisoria entre sistemas
```



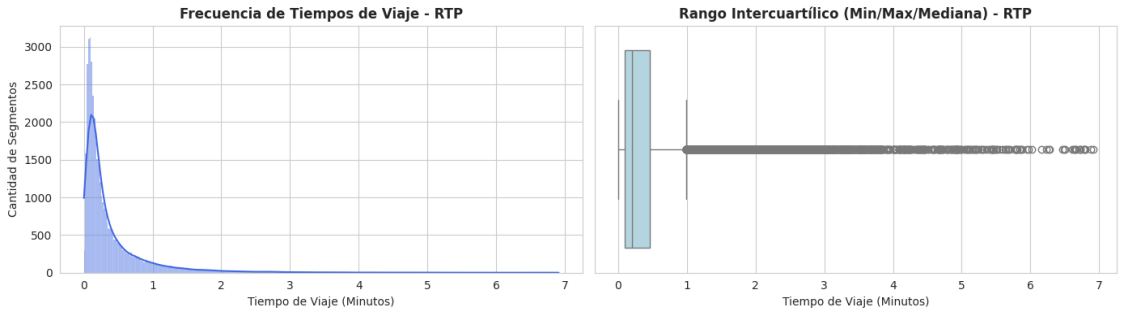
1.3.1 Resumen de Impedancia y Estadísticas: METRO

```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m          95.98    8.12        18.76          31.34        98.71
Friccion_Cf           1.00    1.00         1.00           1.00         1.00
Tiempo_Viaje_min      0.18    0.01         0.03           0.06         0.18

                   Máximo
Distancia_m       2051.11
Friccion_Cf          1.00
Tiempo_Viaje_min     3.31
================================================================================
====================
```



1.3.2 Resumen de Impedancia y Estadísticas: MB

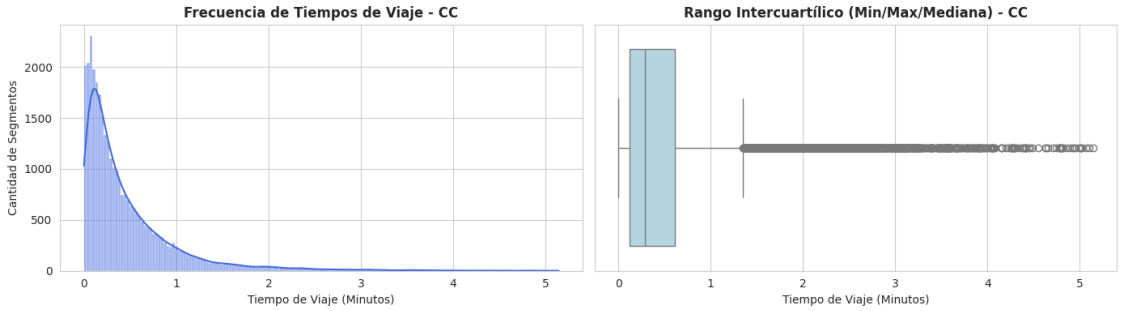
```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m         117.94     0.0        18.72          53.43       171.41
Friccion_Cf           1.00     1.0         1.00           1.00         1.00
Tiempo_Viaje_min      0.44     0.0         0.07           0.18         0.65

                   Máximo
Distancia_m       2812.89
Friccion_Cf          1.00
Tiempo_Viaje_min     9.32
================================================================================
====================
```



1.3.3 Resumen de Impedancia y Estadísticas: RTP

```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m          62.44    0.11         14.7          29.05        67.32
Friccion_Cf           1.40    1.40          1.4           1.40         1.40
Tiempo_Viaje_min      0.42    0.00          0.1           0.20         0.46

                   Máximo
Distancia_m       2758.52
Friccion_Cf          1.40
Tiempo_Viaje_min    18.87
```

==========
==========

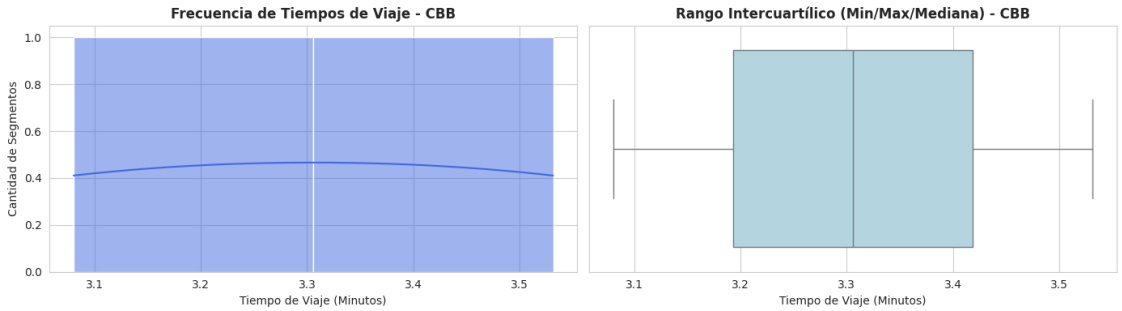


1.3.4 Resumen de Impedancia y Estadísticas: CC

```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m          94.10    0.01        23.61          55.93       120.00
Friccion_Cf           1.40    1.40         1.40           1.40         1.40
Tiempo_Viaje_min      0.48    0.00         0.12           0.29         0.62

                   Máximo
Distancia_m       2046.55
```

```
Friccion_Cf          1.40
Tiempo_Viaje_min     9.92
```

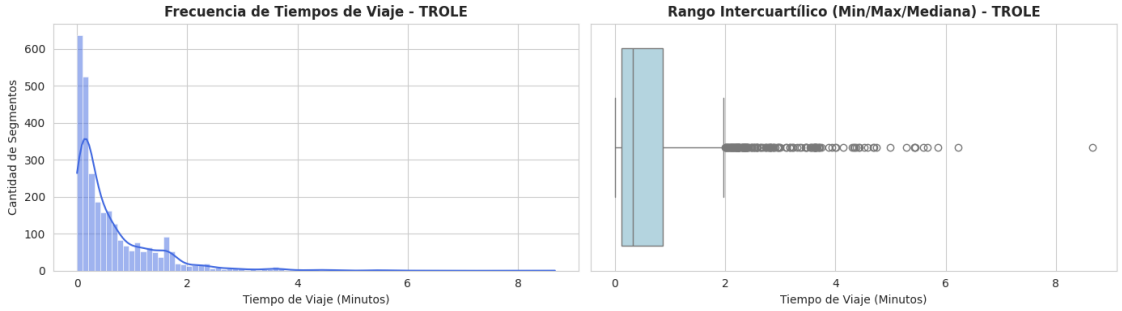
==========
==========



1.3.5 Resumen de Impedancia y Estadísticas: CBB

```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m        1695.77  829.61      1402.43        1692.45      2030.53
Friccion_Cf           1.00    1.00         1.00           1.00         1.00
Tiempo_Viaje_min      6.02    3.08         4.66           6.17         7.14

                   Máximo
Distancia_m       2752.93
Friccion_Cf          1.00
Tiempo_Viaje_min     9.16
```

==========
==========

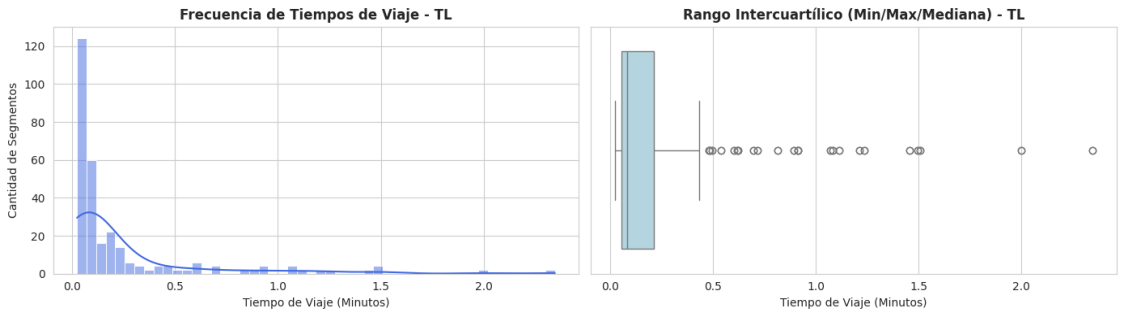


1.3.6 Resumen de Impedancia y Estadísticas: TROLE

```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m         107.99    0.04        20.38          50.74       153.23
```

```
Friccion_Cf           1.40    1.40          1.40           1.40          1.40
Tiempo_Viaje_min      0.67    0.00          0.12           0.33          0.87

                   Máximo
Distancia_m       2430.35
Friccion_Cf          1.40
Tiempo_Viaje_min    14.54
```

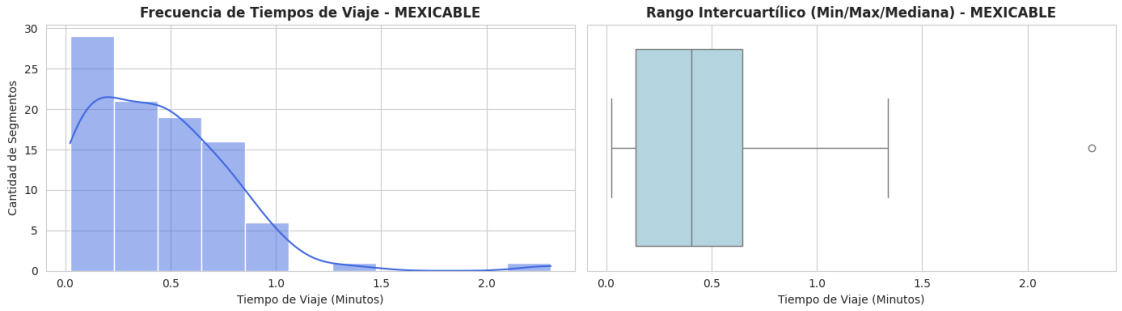
==========
==========



1.3.7 Resumen de Impedancia y Estadísticas: TL

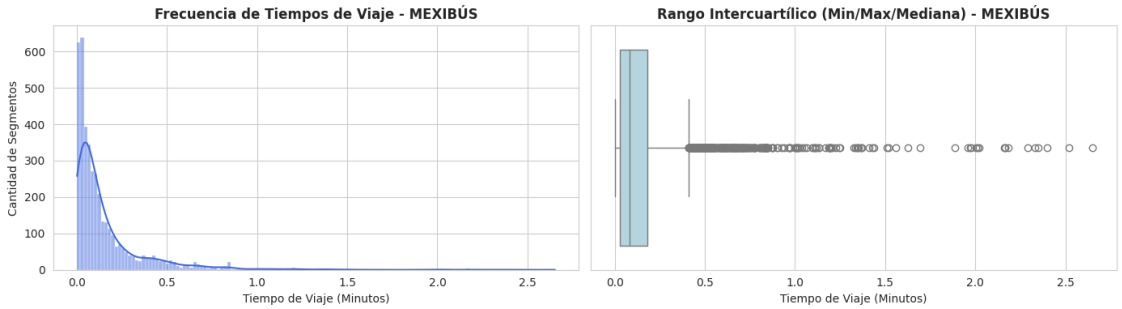
```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m          84.35    8.50        19.01          28.69        73.11
Friccion_Cf           1.00    1.00         1.00           1.00         1.00
Tiempo_Viaje_min      0.25    0.02         0.06           0.08         0.21

                  Máximo
Distancia_m       807.07
Friccion_Cf         1.00
Tiempo_Viaje_min    2.35
```

==========
==========



1.3.8 Resumen de Impedancia y Estadísticas: MEXICABLE

```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m         140.79    7.05        46.98         131.61       210.66
Friccion_Cf           1.00    1.00         1.00           1.00         1.00
Tiempo_Viaje_min      0.44    0.02         0.14           0.40         0.65

                  Máximo
Distancia_m       691.47
Friccion_Cf         1.00
Tiempo_Viaje_min    2.30
================================================================================
====================
```



1.3.9 Resumen de Impedancia y Estadísticas: MEXIBÚS

```
                  Promedio  Mínimo  Cuartil 25%  Mediana (50%)  Cuartil 75%  \
Distancia_m          58.34    0.22        10.89          29.09        65.60
Friccion_Cf           1.16    1.16         1.16           1.16         1.16
Tiempo_Viaje_min      0.16    0.00         0.03           0.08         0.18

                  Máximo
Distancia_m       952.74
Friccion_Cf         1.16
Tiempo_Viaje_min    2.65
================================================================================
====================
```

1.3.10 4. Nota Aclaratoria y Limitaciones del Modelo Base

Los resultados expuestos en esta fase de la investigación representan el **desplazamiento teórico ideal** de la red. Para una correcta interpretación metodológica, se establecen las siguientes delimitaciones:

1. **Naturaleza de las Distancias:** Las longitudes segmentales (D) son aproximaciones geodésicas (calculadas mediante la fórmula de Haversine) basadas en el mapeo topológico de la red de transporte. Representan la distancia espacial entre nodos, no necesariamente el trazo milimétrico de las curvas de las calles.
2. **Tiempos Nominales:** El tiempo de viaje actual asume condiciones operativas constantes. Representa la capacidad estructural del sistema antes de ser sometido a perturbaciones estocásticas en tiempo real (condiciones dinámicas que se abordarán en fases posteriores).
3. **Fundamento de la Fricción Vial:** Los ponderadores utilizados para el cálculo del Coeficiente de Fricción Vial (C_f) se fundamentan en los lineamientos de diseño urbano e interacción modal establecidos en el **Manual de Calles: Diseño Vial para Ciudades Mexicanas (2019)**.
   - *Fuente Oficial:* SEDATU / Gobierno de la CDMX (PDF)